

User Role Can Be Modified in User Profile

Lab Information

Field	Value
Category	Access Control
Difficulty	Apprentice
Vulnerability	Broken Access Control / Mass Assignment

Summary

This lab demonstrates a **Broken Access Control** vulnerability caused by **Mass Assignment**. The application allows users to update their profile information but fails to restrict which parameters can be modified. By adding an undocumented `roleid` parameter to the profile update request, an attacker can elevate their privileges and gain administrative access.

Objective

Modify the user role to gain administrator privileges and delete the user **carlos**.

Attack Scenario

The application provides a profile update feature that allows users to modify their email address.

Although the request does not include a `roleid` parameter by default, the server accepts additional parameters supplied by the client without validating whether they are authorized to be modified.

By adding `roleid=2` to the update request, the attacker successfully changes their role to Administrator and gains access to privileged functionality.

Steps to Reproduce

1. Log in using the provided credentials:
2. Username: `wiener`

3. Password: `peter`
4. Navigate to **My Account**.
5. Update the email address while intercepting the request with Burp Suite.
6. Send the request to **Repeater**.
7. Add the following parameter to the request body:

```
roleid=2
```

1. Send the modified request.
2. Browse to the administrator panel.
3. Delete the user **carlos**.
4. Verify that the lab is solved.

Evidence

Original Request

The profile update request only contained the email parameter.

Modified Request

An additional parameter was injected:

```
roleid=2
```

Privilege Escalation

After the modified request was processed, the application granted administrative privileges and allowed access to the administrator panel.

Security Impact

An attacker can modify sensitive server-side properties that should never be controlled by the client. This can result in privilege escalation, unauthorized access to administrative functionality, and potentially complete compromise of the application.

Root Cause

The application failed to restrict which object properties could be updated by user input. Instead of using an allowlist of permitted fields, the server accepted arbitrary parameters and directly mapped them to internal application objects.

Remediation

- Implement allowlists for user-editable fields.
 - Never bind client-supplied data directly to sensitive server-side objects.
 - Ignore or reject unexpected parameters.
 - Perform server-side authorization checks before applying changes to security-sensitive attributes such as user roles.
-

Pentester's Notes

When testing profile update or account management functionality, always inspect intercepted requests for opportunities to inject undocumented parameters. Fields such as `role`, `roleid`, `isAdmin`, `permission`, `group`, or `accessLevel` are common candidates. Even if they are not present in the original request, adding them manually may reveal Mass Assignment vulnerabilities.

Key Takeaways

- Never assume the client will only submit expected parameters.
- Mass Assignment can expose sensitive object properties to attackers.
- User roles and permissions must never be directly modifiable through client requests.
- Burp Repeater is highly effective for discovering hidden server-side parameters.